Strategies Migration Platform

An AI-triaged enquiry flow, a case portal for staff and clients, and a client app on Android, all built on one API.

NEXUS SOLUTIONS FOR STRATEGIES MIGRATION SERVICES

Marco Alexandre George Hart	ST10462493
Kundai Makuzha	ST10251297
Kaan Dogan	ST10442156
Mbulungeni Kenneth Muthambi	ST10447819
Lekabe Herman Mabalane	ST10278399
Riaad Hercules	ST10258492

Most enquiries were noise, and consultants sorted them by hand

≈80%

of web enquiries were junk that a consultant still had to read and triage manually (client interview).

Enquiries are screened before a person reads them

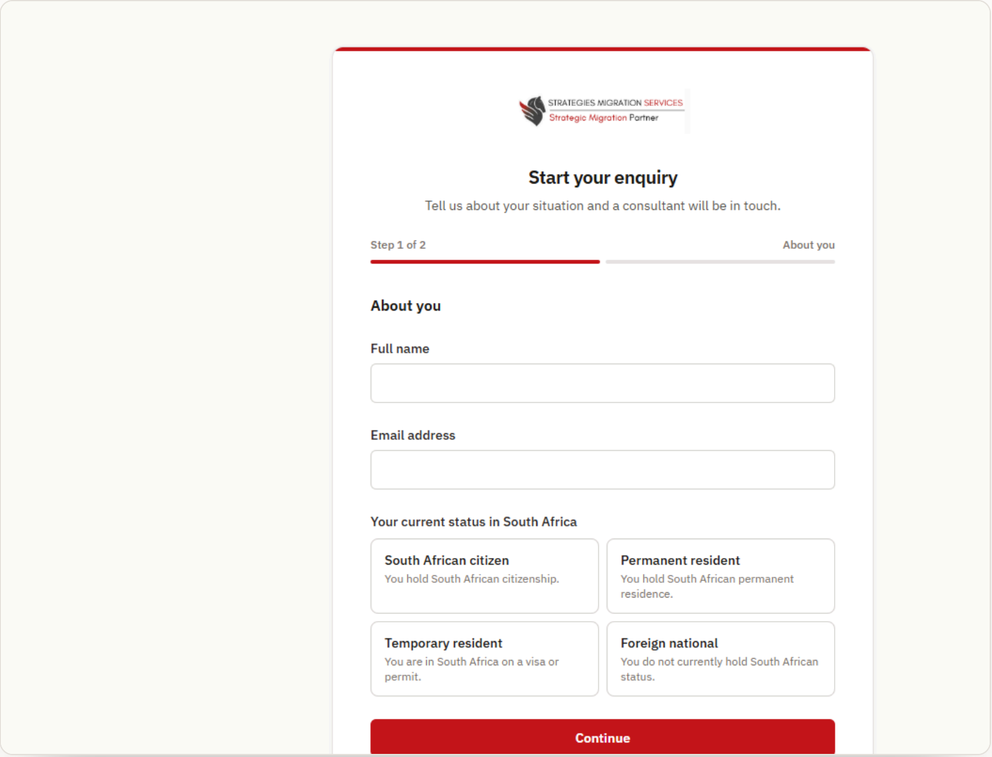
A public form, triaged on the server by a Gemini agent.

Staff work from one portal

Leads, cases, documents, users and an audit trail, by role.

Clients can see where they stand

A five-stage case journey on the web and on Android.

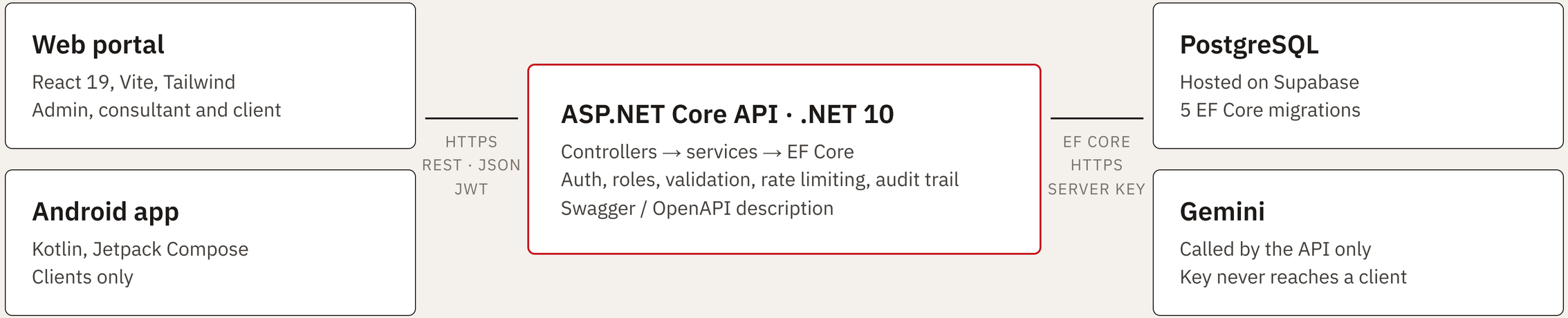


The public enquiry form, step 1

What Task 1 promised, and what we delivered

REQUIREMENT (TASK 1)	STATUS	WHAT WAS BUILT
FR-1 Three roles, JWT sign-in, role portals	● Delivered	Admin, consultant and client, checked on every endpoint; each role lands on its own dashboard
FR-2.1 Conversational intake, four questions	◐ Changed	A structured two-step form with consent; visa category is not asked yet
FR-2.2 / 2.3 AI triage with a safe fallback	● Delivered	Gemini returns a verdict and reason; if it fails, the lead is saved as Needs review
FR-3.1 Admin assigns consultants	◐ Partly	Assigning a consultant to a case works; converting a lead into a case is not built yet
FR-3.2 Rejection needs a written reason	● Delivered	Enforced in the API and the portal; the client sees the note and uploads a new copy
FR-3.3 / 3.4 Checklist and summary agents	○ Not yet	The checklist table exists in the database; both agents share the same agent interface
FR-4.1 Five-stage case lifecycle	● Delivered	Stage changes are recorded in stage history and the audit log
FR-5 Mobile: progress, camera upload, messaging	◐ Partly	Progress, documents and notifications on Android; upload is on the web for now; no messaging
NFR Secrets, rate limiting, soft delete, storage	◐ Mostly	Keys stay on the server, sign-in is rate limited, records are archived; files sit on the API server, not R2

Two clients, one API, and nothing talks to the database directly



Layered. Presentation (web, mobile), business logic (API services), data (EF Core and PostgreSQL).

One contract. Web and mobile use the same endpoints, so rules live in one place.

Replaceable parts. Services sit behind interfaces and are injected.

Small, replaceable services behind a predictable REST API

HOW THE CODE IS BUILT

Interfaces and DI	IAuthService, ITokenService, IPasswordHasherService, IEmailService, IRefreshTokenStore
Strategy pattern	IAgentService, implemented by GeminiAgentService, so the AI provider can be swapped
DTOs	The API contract is separate from the database entities
Logging	ILogger in the auth, email and AI services

HOW THE API BEHAVES

Resources	Auth, leads, cases, documents, users, audit log, notifications, CSV export
Status codes	200, 201, 400, 401, 403, 404 and 429, with one error shape
Lists	Paging, filtering and search run on the server
External service	Gemini triage, with a fallback so no enquiry is lost

From a stranger's enquiry to a tracked case

1

Enquiry

Public form, validated in the browser and again in the API, with POPIA consent.

2

Triage

The API asks Gemini for a verdict and reason. On failure the lead is saved as Needs review.

3

Review

A consultant reads the reason, then confirms or changes the verdict.

4

Case

An admin assigns a consultant. Every stage change is written to history and the audit log.

5

Documents

Approve, or reject with a note. The client uploads a new copy of what was rejected.

THE CASE LIFECYCLE, AS CLIENTS SEE IT ON WEB AND ANDROID



STAGE 1
Initial intake

STAGE 2
Document gathering

STAGE 3
Under review

STAGE 4
Submitted to authority

STAGE 5
Final decision

Every rule is enforced on the server, not just hidden in the interface

Passwords	bcrypt (work factor 11); never stored or logged in plain text
Sessions	JWT access tokens for 60 minutes; refresh tokens for 7 days, rotated and revoked on sign-out
Access	Roles checked on every endpoint; consultants only see their own cases
Brute force	Sign-in limited to 5 attempts a minute per address (429)
Injection and XSS	EF Core parameterised queries, no raw SQL; React escapes all output, no raw HTML
Uploads	PDF, JPG or PNG up to 5 MB, checked on the server
Secrets	User secrets and environment variables; the Gemini key stays in the API
Accountability	An audit trail of who did what, and when

Logged openly in our issue log: the reset-password token (ISS-01), per-account lockout (ISS-05) and development defaults in appsettings (ISS-02).

A relational model, versioned with migrations, hosted on Supabase

12

tables: users, leads, cases, documents, stage history, audit log and reference data

5

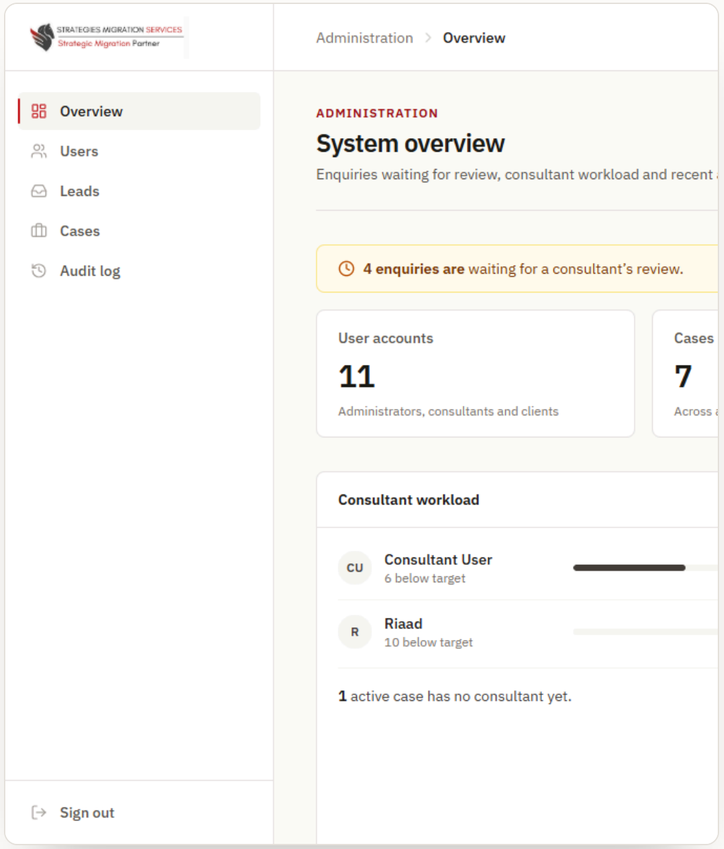
EF Core migrations build the schema from scratch

19

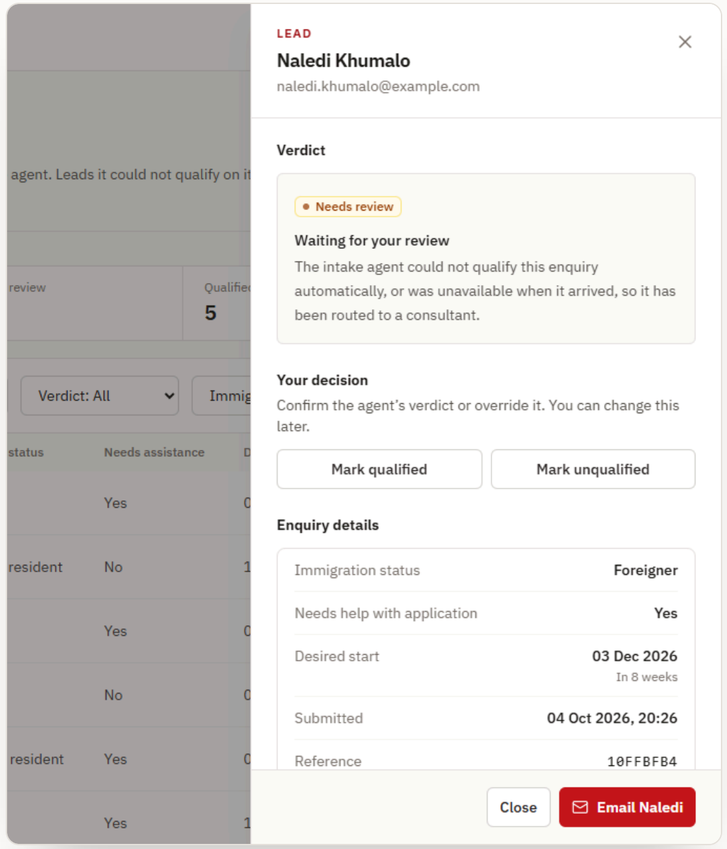
indexes for the lookups the portal runs most

Why relational	Clients, cases, documents and consultants are tightly linked
Integrity	Foreign keys with explicit delete rules, enforced by PostgreSQL
Seed data	11 users, 11 leads, 7 cases; bcrypt-hashed demo passwords
A problem we solved	A schema mismatch broke hosted sign-in; fixed with one tested migration

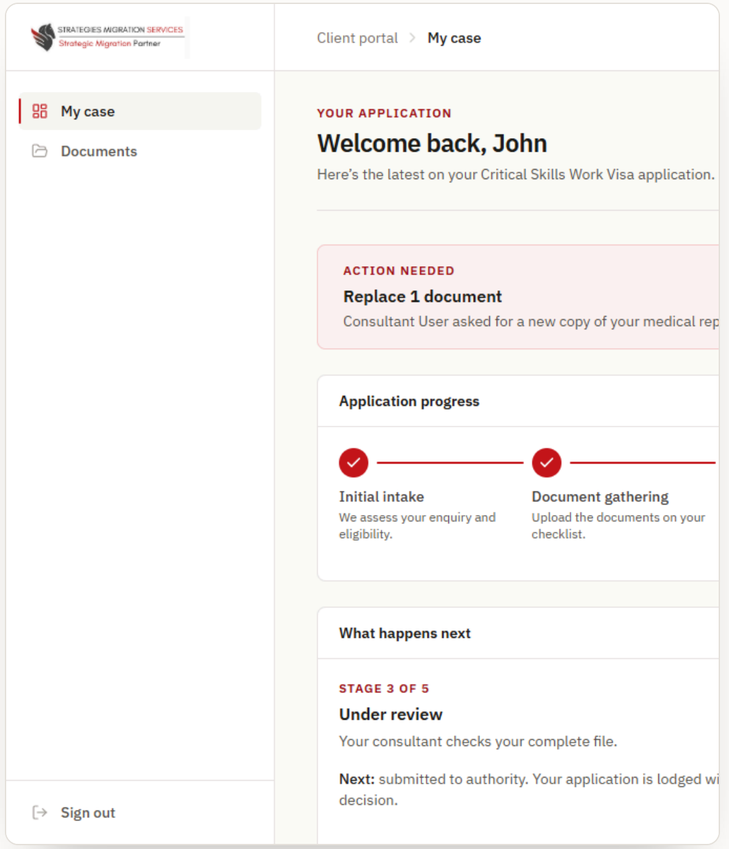
Three roles, three different starting points



Admin · live numbers and consultant workload



Consultant · the agent's verdict and reason



Client · what to do next, and where the case stands

Built for phones and keyboards, not just a large screen

Branding

The client's red and warm greys, used the same way on every screen

Feedback

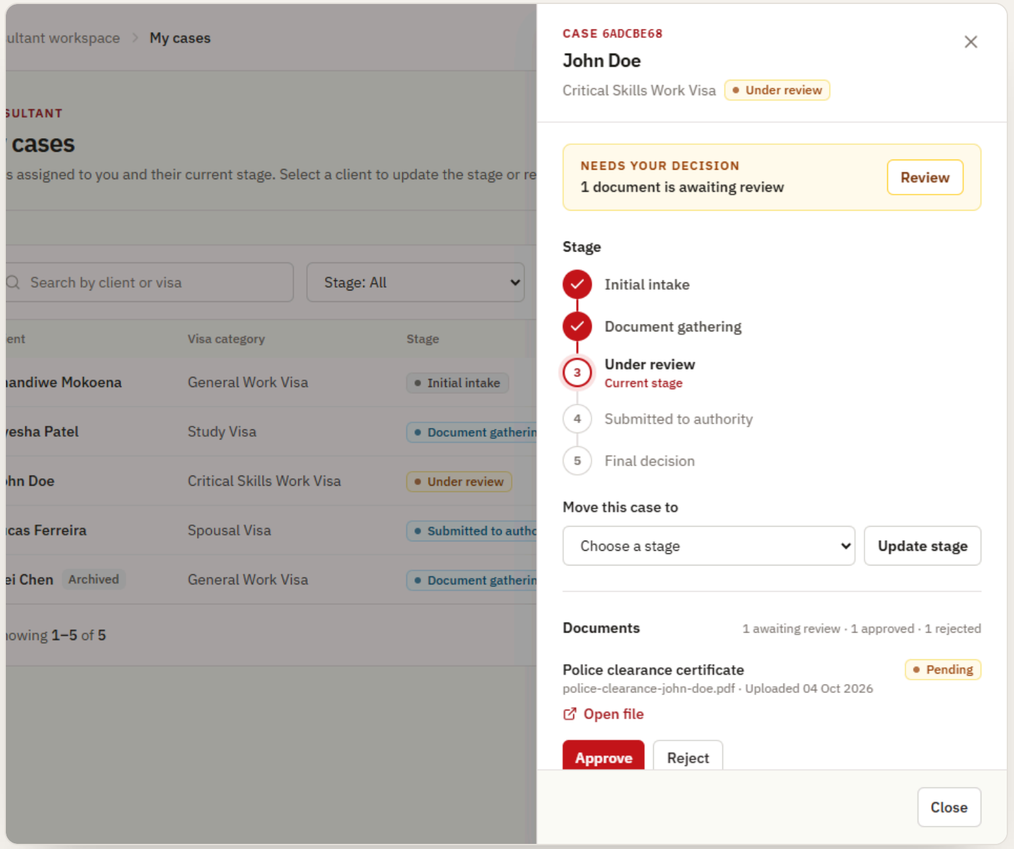
Loading states, toasts, plain-language errors and a safe error screen

Responsive

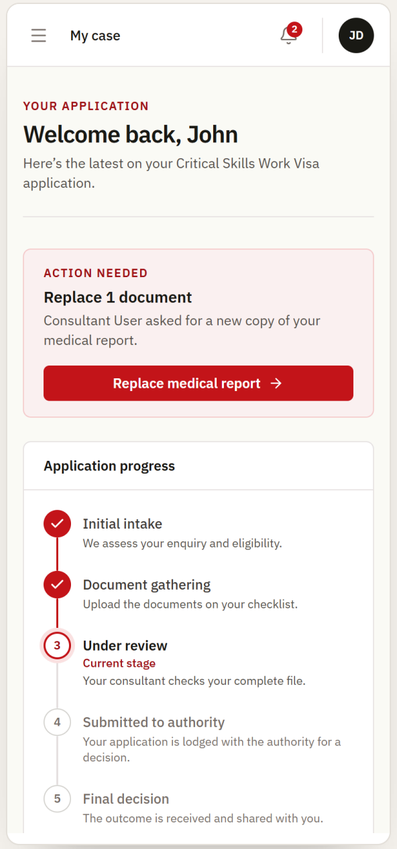
Checked at 1280 and 390 px; tables become cards, navigation becomes a drawer

Accessible

Full keyboard use, visible focus, reduced motion, readable contrast



Consultant case panel · stage and document review



Client · 390 px

The client's case, in their pocket

Native Android

Kotlin and Jetpack Compose, on the same API as the web portal

Only what clients need on the go

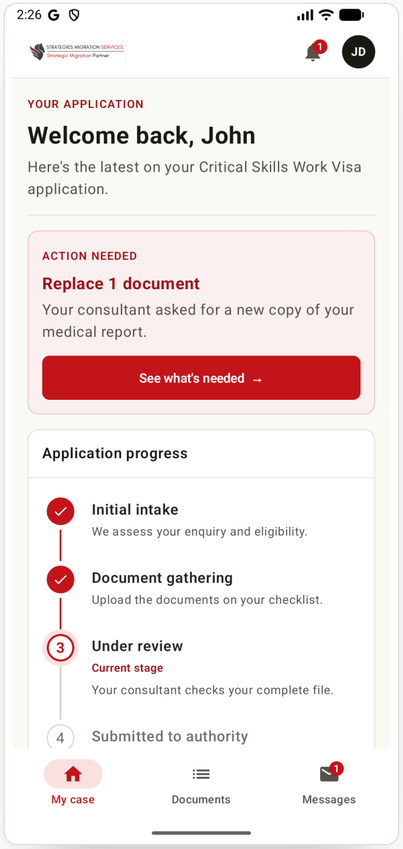
My case, documents and notifications

Clients only

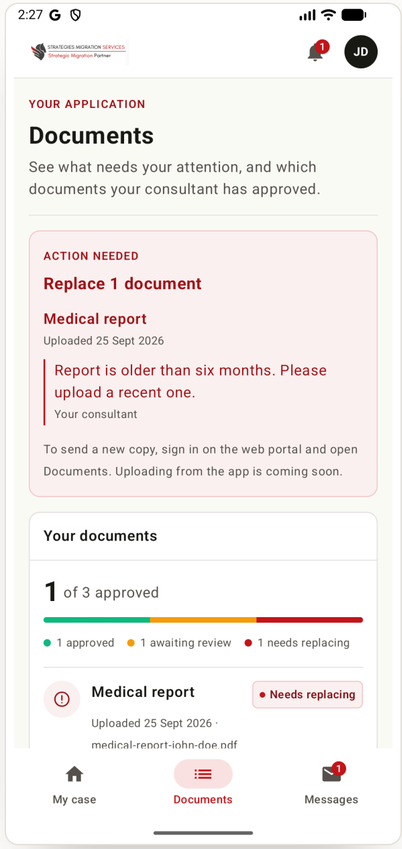
Staff accounts are refused, and signed out on the server too

Tested against the real API

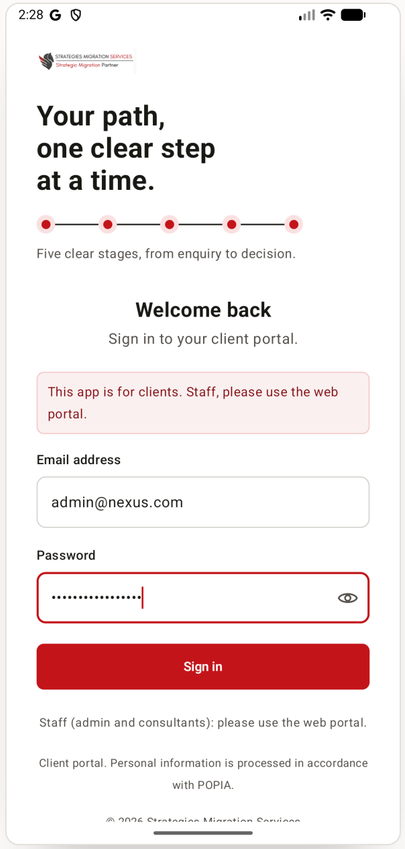
On an emulator, with 19 unit tests passing



My case



Documents



Staff refused

Tested at every layer, with the results written down

185
web tests (Vitest, 19 files), all passing

27
API tests (xUnit), all passing

19
mobile unit tests, all passing

Test plan
TESTING.md maps each requirement to a test with expected and actual results, including auth, AI fallback and mobile

Issue log
ISSUES.md records each bug we found, its owner, and how it was fixed and re-checked

Every change moves through a branch and a pull request

1

Feature branch

One branch per piece of work, for example feature/riaad-docs

2

Pull request

CI builds and tests the change; results show before anyone merges

3

develop

The integration branch; merging here triggers the web deployment workflow

34

pull requests merged

33

branches across six members

2

GitHub Actions workflows

develop is our release branch today; a protected main branch for tagged releases is the next step.

GitHub Actions tests every change and deploys the web portal

CI.YML · ON EVERY PUSH AND PULL REQUEST

.NET job	Restore, build in Release, run the API tests, upload the results
React job	Install, lint, run the web tests, production build

DEPLOY-PAGES.YML · ON PUSH TO DEVELOP

Build	Production build of the web portal
Deploy	Publishes the build to GitHub Pages, hands-off

Status at time of presenting: deployed and live on GitHub Pages

Where each part runs, and why we chose it

PART	PLATFORM	WHY	LIVE ADDRESS
Web portal	GitHub Pages	Free static hosting that deploys straight from our Actions workflow	emgprs.github.io/insy7315-2026-task-2-hartmarcost10462493
API	[host]	Runs the .NET API; secrets set as environment variables on the host	[API URL]
Database	Supabase PostgreSQL	Managed PostgreSQL with backups; connected through the IPv4 session pooler	Private; only the API connects
Android app	APK and emulator	Installable build for demonstration	Uses the hosted API

Every connection uses HTTPS. The database password, JWT key and Gemini key never live in the repository.

What we will show, in this order

1

Enquiry

A visitor submits the public form and the agent triages it.

2

Consultant

Reads the reason, confirms the verdict, reviews a document.

3

Admin

Assigns a consultant, exports cases to CSV, checks the audit log.

4

Client, web

Sees what to do next and uploads a new copy of a rejected document.

5

Client, Android

The same case and documents, on a phone.

Next after Task 2: lead-to-case conversion, case messaging, document checklists per visa type, upload from the phone, and a secure password reset.

THANK YOU

Questions?

Live portal	emgprs.github.io/insy7315-2026-task-2-hartmarcost10462493
API documentation	[API URL]/swagger
Source code	github.com/EMGPRS/insy7315-2026-task-2-hartmarcost10462493
Demo video	youtu.be/aw8wSutQ7Z8